CS527 Lab - Introduction to OpenMP (Arrays and Matrices)

Five small C++ programs. Each one has a serial version and an OpenMP version of the same function, checks that both give the same answer, and prints a timing table.

Task	File	What it does
1	`src/task1_vecadd.cpp`	Vector addition with `parallel for`
2	`src/task2_sum.cpp`	Array sum: broken naive version, `reduction`, `critical`
3	`src/task3_maxmin.cpp`	Max / min value and its index
4	`src/task4_matadd.cpp`	Matrix addition: outer loop vs `collapse(2)`
5	`src/task5_matmul.cpp`	Matrix multiplication: `i` loop vs `collapse(2)`, plus `i,k,j` bonus

The exact program output that the tables below are taken from is saved in the `results/` folder.

Build and run

```
make            (on Windows with MinGW: mingw32-make)
./task1
./task2
./task3
./task4
./task5
```

or compile one file by hand:

```
g++ -fopenmp -O2 -Wall src/task1_vecadd.cpp -o task1
```

All five files compile with `-fopenmp -O2 -Wall` without any warning. The programs take no arguments; the sizes and thread counts are written in `main`.

My setup

- CPU: AMD Ryzen 7 5800H, 8 cores / 16 logical CPUs
- RAM: 16 GB (2 x 8 GB DDR4-3200)
- OS: Windows 11, compiler: g++ 13.2.0 (MinGW-w64), OpenMP 4.5
- **Thread count method:** `omp_set_num_threads(n)` in the code. Each program loops over 1, 2, 4, 8 and 16 threads (16 = all logical CPUs).

How I measured time

- Time is measured with `omp_get_wtime()`.
- On my machine `omp_get_wtime()` only changes every 1 ms (`omp_get_wtick()` returns 0.001). Many of the runs are shorter than that, so every function is called `repeat` times in a loop and I report the **average time of one call**. The `critical` versions are the exception: they are so slow that they are run only once.
- Speedup = `T_serial / T_parallel`. Efficiency = speedup / threads.
- All numbers are from one run of each program on a laptop that was also running other things (a background OneDrive sync was using about one CPU core the whole time). So the numbers are noisy. The parallel version with **1 thread** does exactly the same work as the serial function, but in some tables the two differ by up to about 30%. When I timed the two one after the other several times (Task 4, N = 4096) the same call gave anything from 18 to 22 ms, with no pattern. So a "speedup" of up to about 1.3 in these tables can be just noise and should not be read as a real gain.

Task 1 - Vector Addition

`c[i] = a[i] + b[i]` with `a[i] = i` and `b[i] = 2i`. These are whole numbers, so the addition is exact and the serial and parallel results are compared element by element with `==` (no tolerance). **All runs were identical to the serial result.**

Time in ms (speedup in brackets):

Threads	N = 10^5	N = 10^6	N = 10^7	N = 10^8
serial	0.0531	1.535	13.55	134.5
1	0.0418 (1.27)	1.197 (1.28)	14.13 (0.96)	133.2 (1.01)
2	0.0550 (0.97)	0.896 (1.71)	12.69 (1.07)	105.4 (1.28)
4	0.0477 (1.11)	0.816 (1.88)	10.92 (1.24)	118.1 (1.14)
8	0.0582 (0.91)	0.793 (1.94)	12.25 (1.11)	119.2 (1.13)
16	0.0958 (0.55)	0.766 (2.00)	12.62 (1.07)	124.2 (1.08)

Speedup for the largest N (10^8):

Threads	1	2	4	8	16
Speedup	1.01	1.28	1.14	1.13	1.08

Explanation. Vector addition does almost no computation per element, it only reads two arrays and writes a third. For large N the speed is decided by how

fast data can be moved between RAM and the CPU, and one thread already uses most of that. So adding threads gives almost no speedup on my machine.

At what array size does parallelization start paying off? Why not for small N? For N = 10^5 it does not pay off: 2 and 4 threads are about the same as serial and 8 / 16 threads are slower. Starting the parallel region and waking up the threads costs some tens of microseconds (Task 4 shows this cost directly), and the whole job is only about 50 microseconds, so the overhead is as big as the work. At N = 10^6 there is a gain (about 1.7x to 2x in this run). For N = 10^7 and 10^8 the arrays are far bigger than the CPU cache (16 MB) and the gain is small again (1.1x to 1.3x) because of the memory limit described above. So on my machine it only really helps around N = 10^6.

Does speedup keep increasing linearly with thread count? No. The best speedup is 2.0 with 16 threads (N = 10^6), which is far from 16x, and for the large arrays it stays between 1.1 and 1.3 no matter how many threads are used. All threads share the same memory, so when the memory is the bottleneck more threads cannot make it faster. Also 16 threads are only 8 real cores (2 logical CPUs per core).

Task 2 - Sum of Array Elements

The array is `a[i] = i + 1`, so the correct sum is `N(N+1)/2`. I used `long long` instead of `long` because on Windows `long` is 32 bits and this sum does not fit.

Step 1: the naive version is wrong

`sum_naive_parallel_BROKEN` (plain `parallel for`, every thread does `sum += a[i]`), N = 10^7, 8 threads, run 10 times:

```
Correct answer = 50000005000000
Run  1: sum =  11718750625000   WRONG
Run  2: sum =   7031250625000   WRONG
Run  3: sum =   7031250625000   WRONG
Run  4: sum =  11718750625000   WRONG
Run  5: sum =  10156250625000   WRONG
Run  6: sum =  10156250625000   WRONG
Run  7: sum =    781250625000   WRONG
Run  8: sum =   8593750625000   WRONG
Run  9: sum =   3906250625000   WRONG
Run 10: sum =   2343750625000   WRONG
```

All 10 runs are wrong, and the wrong value changes from run to run.

Steps 2-4: `reduction(+:sum)`

The reduction result was compared with the serial sum after **every** call (100 calls for N = 10^7, 10 calls for N = 10^8) and it matched every time. The serial sum also equals `N(N+1)/2`.

Time in ms (speedup in brackets):

Threads	N = 10^7	N = 10^8
serial	3.88	42.8
1	3.88 (1.00)	45.6 (0.94)
2	2.50 (1.55)	26.8 (1.60)
4	1.87 (2.07)	20.2 (2.12)
8	2.08 (1.87)	21.6 (1.98)
16	2.52 (1.54)	25.3 (1.69)

Explanation. `reduction` gives a correct answer and about 2x speedup with 4 threads. After that it stops improving, because summing is also a very light operation and the threads are again waiting for data from memory (10^8 `long long` values are 800 MB; 20.2 ms means about 40 GB per second, which is close to what this RAM can deliver).

`critical` instead of `reduction` (N = 10^7, one run each)

Threads	critical (ms)	reduction (ms)	Correct?
1	44	3.88	yes
2	97	2.50	yes
4	101	1.87	yes
8	190	2.08	yes
16	241	2.52	yes

Why does the naive version give wrong / inconsistent answers? `sum` is one variable shared by all threads, and `sum += a[i]` is really three steps: read `sum`, add, write `sum` back. Two threads can read the same old value, both add their own element, and both write back, so one of the two additions is lost. This is a data race, and how many updates are lost depends on the timing of the threads, so the answer is different on different runs. In my runs the effect is very clear: every wrong value is exactly the sum of the part of the array that **one** thread handled (for example 781250625000 = 1 + 2 + ... + 1250000, which is thread 0's part). With `-O2` each thread keeps its running total in a register and writes it into `sum` at the end, so whichever thread writes last overwrites the work of the other seven.

Would `critical` fix correctness? What about performance? Yes, it fixes correctness: only one thread at a time can execute `sum += a[i]`, so no update is lost (all rows above are correct). But the performance is very bad. The threads cannot add in parallel any more, and every single addition has to take and release a lock. It is 11x slower than the serial loop even with 1 thread (44 ms vs 3.88 ms), and it gets *slower* when threads are added because they fight for the lock (241 ms with 16 threads, about 60x slower than serial and almost 100x slower than `reduction`). `reduction` avoids this because each thread adds into its own private copy and the copies are combined only once at the end.

Task 3 - Maximum and Minimum of an Array

The array has random values from 0 to 9999, and I put one clear maximum (99999 at index N/3) and one clear minimum (-5 at index 2N/3) so the correct answer is known. Three methods are implemented, each for max and for min:

1. `reduction(max:max_val)` / `reduction(min:min_val)` - value only
2. a `critical` section updating a shared `(best_val, best_idx)` - value and index
3. a manual reduction - each thread finds its own `(val, idx)` in private variables, stores it in its slot of a per-thread array, and a serial loop picks the overall best - value and index

Every result was compared with the serial version (value and index) and **all of them matched**.

Each time below is for finding the max **and** the min (one call each), in ms.

N = 10^7 (serial: 10.89 ms, max = 99999 at index 3333333, min = -5 at index 6666666)

Threads	reduction (value only)	manual (value + index)	manual speedup	critical (value + index)
1	10.54	10.69	1.02	94
2	5.72	5.85	1.86	208
4	3.10	3.31	3.29	212
8	1.87	2.77	3.93	338
16	2.33	2.81	3.88	471

N = 10^8 (serial: 105.8 ms, max = 99999 at index 33333333, min = -5 at index 66666666)

Threads	reduction (value only)	manual (value + index)	manual speedup	critical (value + index)
1	105.4	106.1	1.00	not run
2	54.3	54.9	1.93	not run
4	28.6	28.1	3.77	not run
8	21.4	22.3	4.74	not run
16	27.0	26.9	3.93	not run

The `critical` version was not run for N = 10^8 because it would take several seconds per row and the N = 10^7 result already shows the problem.

Explanation. Here the speedup is good up to 4 threads (about 3.8x for N = 10^8), which is much better than Tasks 1 and 2. Each element needs a comparison, so there is more real work per element, and nothing is written to memory. After 4 to 8 threads it stops improving, again because of the memory speed (two scans of a 400 MB array in about 21 ms).

Why can't you write `reduction(max: best_idx)` to get the index? `reduction(max: x)` keeps the largest value of `x` itself. If `x` is the index, the result is just the largest *index* that any thread stored, not the index of the largest *value*. The index has to stay together with its value: when two threads' results are combined, we must compare the values and then take the index that belongs to the winning value. A plain `reduction` works on one variable with one operator, so it cannot do that. This needs a pair `(value, index)` that is combined together, which is what the `critical` and the manual versions do.

`critical` vs manual per-thread reduction - which is faster and why? The manual reduction is far faster: for N = 10^7 with 8 threads it takes 2.77 ms against 338 ms for `critical` (about 120x). The `critical` version takes a lock for **every element**, so the threads run one at a time and also pay for the lock; it is about 9x slower than serial with 1 thread and gets worse with more threads. In the manual version each thread works only on its own private variables with no lock at all, and the shared array is touched once per thread at the very end, so the threads really run in parallel.

Note: my test data has only one maximum and one minimum. If the maximum appeared several times, the `critical` version could return the index of any of them (whichever thread got there first), so a tie rule would be needed.

Task 4 - Matrix Addition

Storage: 2D arrays (an array of row pointers, each row allocated with `new`), so elements are written as `A[i][j]`.

`A[i][j] = i + j`, `B[i][j] = i - j`. Both parallel versions were compared with the serial result with `==` and **both matched** for every thread count.

N = 8, time per call in **microseconds** (average of 100000 calls):

Threads	outer loop only	collapse(2)
serial	0.020	-
1	4.30	4.26
2	20.18	20.50
4	28.25	28.32
8	41.09	40.73
16	67.22	66.66

N = 4096, time per call in **milliseconds** (average of 10 calls), speedup in brackets:

Threads	outer loop only	collapse(2)
serial	22.5	-
1	16.4 (1.37)	17.9 (1.26)
2	16.1 (1.40)	16.7 (1.35)
4	18.4 (1.22)	18.1 (1.24)
8	19.9 (1.13)	20.6 (1.09)
16	21.2 (1.06)	21.3 (1.06)

Explanation. For N = 8 the parallel versions are hundreds of times slower than serial: adding 64 numbers takes 0.02 microseconds, but starting a parallel region costs 4 to 67 microseconds. For N = 4096 there is practically no real gain: the largest speedup is 1.40, and 1 thread already shows 1.37, which can only be noise (see "How I measured time"). The reason is the same as in Task 1: matrix addition only moves data, so the memory is the limit.

For small N, does `collapse(2)` help compared to the outer loop only? In theory it gives more pieces to share out. With the outer loop only there are just 8 iterations (the 8 rows), so with 16 threads only 8 threads can get a row and the other 8 get nothing. With `collapse(2)` the two loops become one loop of 8 x 8 = 64 iterations, so all 16 threads get 4 elements each. With 8 or fewer threads it makes no difference to the sharing, because 8 rows are already enough to give every thread one row. In practice it did not help in my measurements (67.22 vs 66.66 microseconds with 16 threads): the work is so small that the cost of the parallel region decides the time, not how the 64 additions are divided.

For large N, does `collapse(2)` still make a difference? No measurable difference; the two columns are the same within the run-to-run noise. With N = 4096 the outer loop alone already has 4096 rows, which is far more than 16 threads, so every thread is kept busy with an equal share. `collapse(2)` has nothing to improve.

Task 5 - Matrix-Matrix Multiplication (Naive)

Storage: 2D arrays (array of row pointers), same as Task 4. `A` and `B` are filled with random values between 0 and 1.

In the `i,j,k` versions the sum for one element is collected in a local variable `sum` and written to `C[i][j]` once (as the handout's note suggests), in the serial and in both parallel versions. `C` is still zero-initialized in `main`. All parallel results were compared with the serial result using a tolerance of `1e-6` and **all matched**.

Loop order i, j, k

Time in ms. S = speedup, E = efficiency (speedup / threads).

N = 256 (serial 11.95 ms)

Threads	`i` loop (ms)	S	E	collapse(2) (ms)	S	E
1	11.95	1.00	1.00	12.15	0.98	0.98
2	6.75	1.77	0.89	6.50	1.84	0.92
4	3.90	3.06	0.77	3.55	3.37	0.84
8	2.40	4.98	0.62	2.95	4.05	0.51
16	2.15	5.56	0.35	2.15	5.56	0.35

N = 512 (serial 107.00 ms)

Threads	`i` loop (ms)	S	E	collapse(2) (ms)	S	E
1	100.00	1.07	1.07	101.40	1.06	1.06
2	50.80	2.11	1.05	51.40	2.08	1.04
4	26.20	4.08	1.02	27.00	3.96	0.99
8	18.00	5.94	0.74	18.20	5.88	0.73
16	13.40	7.99	0.50	14.40	7.43	0.46

N = 1024 (serial 2032.00 ms)

Threads	`i` loop (ms)	S	E	collapse(2) (ms)	S	E
1	1825.50	1.11	1.11	2225.00	0.91	0.91
2	998.50	2.04	1.02	1017.50	2.00	1.00
4	495.50	4.10	1.03	479.00	4.24	1.06
8	344.00	5.91	0.74	353.50	5.75	0.72
16	328.00	6.20	0.39	351.50	5.78	0.36

Explanation. Matrix multiplication does a lot of computation per element of `C` (N multiplications), so unlike Tasks 1 and 4 it scales well: about 2x with 2

threads and about 4x with 4 threads for N = 512 and 1024 (efficiency close to 1). With 8 threads the speedup is about 5 to 6 (efficiency 0.5 to 0.75), and 16 threads add only a little more because they are 8 real cores. I think the drop at 8 threads comes from all cores sharing the same cache and memory, and from the background process that was using one core. The efficiencies slightly above 1 (up to 1.11) are within the measurement noise, so I do not treat them as real. N = 256 scales a bit worse than the larger sizes; probably because one call takes only a few milliseconds, so small fixed costs (waking up the threads) and noise weigh more.

Bonus: loop order i, k, j

N	serial `i,j,k` (ms)	serial `i,k,j` (ms)	`i,k,j` is faster by
256	11.95	11.65	about the same
512	107.00	59.00	1.8x
1024	2032.00	503.00	4.0x

Parallel `i` loop with `i,k,j` order, time in ms (speedup against serial `i,k,j`, efficiency):

Threads	N = 256	N = 512	N = 1024
1	12.55 (0.93, 0.93)	56.00 (1.05, 1.05)	529.50 (0.95, 0.95)
2	5.65 (2.06, 1.03)	28.80 (2.05, 1.02)	240.00 (2.10, 1.05)
4	2.85 (4.09, 1.02)	16.80 (3.51, 0.88)	126.00 (3.99, 1.00)
8	2.10 (5.55, 0.69)	10.60 (5.57, 0.70)	96.50 (5.21, 0.65)
16	1.45 (8.03, 0.50)	9.20 (6.41, 0.40)	90.00 (5.59, 0.35)

Does the loop order change the serial time? Yes, for the larger sizes. In `i,j,k` the inner loop reads `B[k][j]` for k = 0, 1, 2, ..., which goes *down a column* of `B`, so every step lands in a different row (a different part of memory). In `i,k,j` the inner loop goes along a row of `B` and a row of `C`, so it reads memory in order and the cache works well. For N = 256 there is no real difference, for N = 512 it is 1.8x and for N = 1024 it is 4x. I think it grows like this because the matrices get too big for the cache (at N = 1024 the three matrices together are 24 MB and the cache is 16 MB), so the column-wise reads keep missing the cache.

Does it change how well the parallel version scales? Not much: the shape is the same as for `i,j,k` (about 2x with 2 threads, about 4x with 4, then flatter with 8 and 16). What changes is the absolute time: the best time for N = 1024 is 90 ms with `i,k,j` against 328 ms with `i,j,k`.

Questions

Why is it safe to parallelize the `i` loop directly? Each iteration of the `i` loop writes only to row `i` of `C`, and different threads get different values of `i`, so two threads never write to the same element. `A` and `B` are only read. The variables `j`, `k` and `sum` are declared inside the loop, so every thread has its own copy. The whole `k` summation for one `C[i][j]` is done by one thread from start to end, so there is nothing shared to race on.

What happens if the `k` loop is parallelized with a plain `parallel for`? Then for one fixed `(i, j)` all the threads add into the *same* place (`C[i][j]`, or the shared `sum`). `+=` is read, add, write, so two threads can read the same old value and one of the two additions is lost, exactly like the naive sum in Task 2. The result would be wrong (too small) and different from run to run. It would need `reduction(+:sum)` to be correct, and it would also be slow because a parallel region would be started N x N times.

Why not collapse the `k` loop as well? `collapse(3)` would make one big loop over all `(i, j, k)` and share those out, so the different `k` terms of the *same* `C[i][j]` could go to different threads. That is the same race as above. Also the statements `sum = 0` and `C[i][j] = sum` sit between the `j` and `k` loops, and `collapse` needs the loops to be directly inside each other with nothing in between. So only `i` and `j` are collapsed, and each `(i, j)` element is computed completely by one thread.

`parallel for` on `i` vs `collapse(2)` - when does `collapse(2)` matter most? For my sizes there is no real difference (the two columns are within noise), because N = 256 to 1024 is much larger than 16 threads, so the `i` loop alone gives every thread many rows. `collapse(2)` should matter most when N is small compared to the thread count: if N is less than the number of threads, some threads get no row at all, and if N is only a little larger the rows cannot be divided evenly. With `collapse(2)` there are N x N pieces to share, so the work can be balanced. This is the same situation as N = 8 with 16 threads in Task 4.

Why is a local `sum` better than `C[i][j] += ...` inside the `k` loop? With `C[i][j] += ...` the program has to read and write memory in every `k` step, because the compiler cannot be sure that `C` is a different piece of memory from `A` and `B`. A local `sum` can stay in a CPU register for the whole `k` loop and memory is written only once.